



JAGANNATH UNIVERSITY

Department of Computer Science and Engineering (CSE)

DIGITAL IMAGE PROCESSING (DIP) PROJECT REPORT

Fruit Classification Using Convolutional Neural Network (CNN)

Prepared By

Anowar Hossain

ID: M260105027

MSc in Computer Science and Engineering (CSE)

Jagannath University

Course Title : Digital Image Processing (DIP)

Project Title : Fruit Classification System Using Deep Learning

Submitted To

Dr. Md. Zulfiker Mahmud

Professor

Department of Computer Science and Engineering (CSE)

Jagannath University

Submission Date : 25 June 2026

1. Introduction

Project Title

Fruit Classification Using Convolutional Neural Network (CNN)

Brief Project Overview

This project presents an image classification system that automatically identifies fruits from images using Deep Learning techniques. A **Convolutional Neural Network (CNN)** model is developed and trained to classify four fruit categories: **Apple, Avocado, Blueberry, and Capsicum.**

The system accepts an uploaded image as input and processes it through the trained CNN model to determine the corresponding fruit class. The prediction result is then displayed through a user-friendly **Graphical User Interface (GUI)**, allowing users to interact with the system easily. This project demonstrates the practical application of Deep Learning and Computer Vision techniques in automated fruit recognition and classification tasks.

Objective

- To develop an automated fruit recognition system.
- To classify fruit images into predefined categories.
- To apply Digital Image Processing and Deep Learning techniques.
- To create a user-friendly interface for fruit identification.

Applications

- Smart agriculture
- Automated fruit sorting systems
- Grocery inventory management
- Educational and research purposes

2. Dataset Description

Dataset Name / Source

Fruit Image Dataset collected from:

- Kaggle Fruit Dataset
- Public image repositories

Classes

Class ID	Fruit Name
1	Apple
2	Avocado
3	Blueberry
4	Capsicum

Dataset Statistics

Description	Count
Total Images	381
Training Images	254
Testing Images	9
Validation Images	118

Image Size

- 128 × 128 pixels
- RGB color images

Preprocessing

- Image resizing (128×128)
- Pixel normalization (0–1)
- Data augmentation:
 - Rotation
 - Zoom
 - Horizontal flip
 - Width/Height shift

3. Methodology

Data Collection

Fruit images were collected from publicly available datasets. Images were organized into four categories.

Image Preprocessing

1. Image loading
2. Resizing to 128×128 pixels

3. Normalization
4. Dataset splitting
5. Data augmentation

CNN Architecture

Layer	Details
Input Layer	128×128×3
Conv2D	32 Filters
BatchNormalization	Yes
MaxPooling2D	3×3
Conv2D	64 Filters
BatchNormalization	Yes
MaxPooling2D	2×2
Flatten	Yes
Dense	128 Neurons
Dropout	0.5
Output Layer	4 Classes (Softmax)

Algorithm Used

- Convolutional Neural Network (CNN)

Tools and Libraries

Tool	Purpose
Python	Programming Language
TensorFlow	Deep Learning
Keras	Model Building
NumPy	Numerical Processing
Matplotlib	Visualization

Tool	Purpose
Scikit-Learn	Evaluation Metrics
Gradio	GUI Development

4. Result with Code

Model Training Code

01.Zip file extract

```
import zipfile

zip_ref = zipfile.ZipFile('H:\\project\\fruits_dip.zip', 'r')

zip_ref.extractall('H:\\project\\fruits_dip')

zip_ref.close()

print("Dataset extracted successfully")
```

Dataset extracted successfully

02.Preparing Dataset

```
from tensorflow import keras

# generators
train_ds = keras.utils.image_dataset_from_directory(
    directory=r'H:\project\fruits_dip\train',
    labels = 'inferred',
    label_mode = 'int',
    batch_size = 32,
    image_size = (128,128),
    color_mode='rgb',
    shuffle=True
)

val_ds = keras.utils.image_dataset_from_directory(
    directory=r'H:\project\fruits_dip\valid',
    labels = 'inferred',
    label_mode = 'int',
    batch_size = 32,
    image_size = (128,128),
```

```
        color_mode='rgb',  
        shuffle=True  
    )
```

Found 254 files belonging to 4 classes.

Found 188 files belonging to 4 classes.

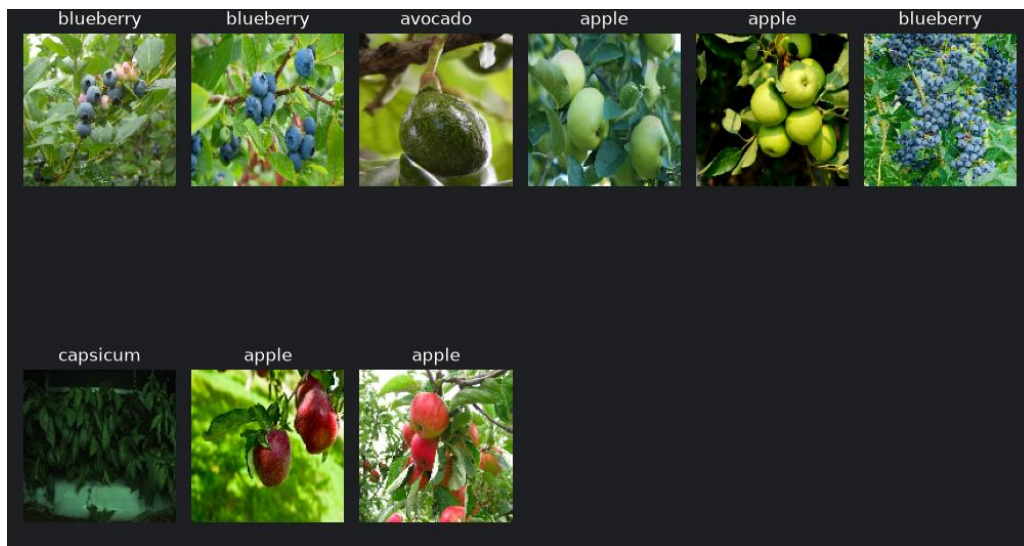
03. Trained Name

- `print(train_ds.class_names)`

['apple', 'avocado', 'blueberry', 'capsicum']

04. Image Visualization

```
import matplotlib.pyplot as plt  
import numpy as np  
  
# Get class names  
class_names = train_ds.class_names  
  
# Extract one batch from the dataset  
for images, labels in train_ds.take(1):  
    images = images.numpy().astype("uint8")  
    labels = labels.numpy()  
  
    # Set up subplot grid  
    plt.figure(figsize=(10, 10))  
  
    for i in range(9): # First 9 images  
        ax = plt.subplot(3, 6, i + 1)  
        plt.imshow(images[i])  
        plt.title(class_names[labels[i]])  
        plt.axis("off")  
  
plt.tight_layout()  
plt.show()
```



05. Model Building

```
from keras import Sequential
from keras.layers import
Dense, Conv2D, MaxPooling2D, Flatten, BatchNormalization, Input, Dropout

model = Sequential()
model.add(Input(shape=(128, 128, 3)))
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', padding="same"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3), padding='same'))

model.add(Conv2D(64, kernel_size=(3, 3), activation='relu', padding="same"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3), padding='same'))

model.add(Conv2D(128, kernel_size=(3, 3), activation='relu', padding="same"))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(3, 3), padding='same'))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(4, activation='softmax'))
```

```
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 128, 128, 32)	896
batch_normalization_12 (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d_12 (MaxPooling2D)	(None, 43, 43, 32)	0
conv2d_13 (Conv2D)	(None, 43, 43, 64)	18,496
batch_normalization_13 (BatchNormalization)	(None, 43, 43, 64)	256
max_pooling2d_13 (MaxPooling2D)	(None, 15, 15, 64)	0
conv2d_14 (Conv2D)	(None, 15, 15, 128)	73,856
batch_normalization_14 (BatchNormalization)	(None, 15, 15, 128)	512
max_pooling2d_14 (MaxPooling2D)	(None, 5, 5, 128)	0
flatten_4 (Flatten)	(None, 3200)	0
dense_12 (Dense)	(None, 128)	409,728
dropout_8 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 64)	8,256
dropout_9 (Dropout)	(None, 64)	0
dense_14 (Dense)	(None, 4)	260

Total params: 512,388 (1.95 MB)

Trainable params: 511,940 (1.95 MB)

Non-trainable params: 448 (1.75 KB)

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```


06. Model Training

```
epochs=20  
history = model.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Epoch 1/20

8/8 ————— 6s 356ms/step - accuracy: 0.3780 -
loss: 3.5339 - val_accuracy: 0.4149 - val_loss: 4.5558

Epoch 2/20

8/8 ————— 3s 321ms/step - accuracy: 0.4409 -
loss: 2.3598 - val_accuracy: 0.3723 - val_loss: 2.5034

Epoch 3/20

8/8 ————— 3s 321ms/step - accuracy: 0.5315 -
loss: 1.3745 - val_accuracy: 0.4468 - val_loss: 1.6406

Epoch 4/20

8/8 ————— 3s 319ms/step - accuracy: 0.5354 -
loss: 1.2701 - val_accuracy: 0.5053 - val_loss: 1.0944

Epoch 5/20

8/8 ————— 3s 324ms/step - accuracy: 0.6142 -
loss: 1.0178 - val_accuracy: 0.4468 - val_loss: 1.5539

Epoch 6/20

8/8 ————— 3s 318ms/step - accuracy: 0.6260 -
loss: 0.8589 - val_accuracy: 0.4309 - val_loss: 1.5895

Epoch 7/20

8/8 ————— 3s 322ms/step - accuracy: 0.5709 -
loss: 0.9531 - val_accuracy: 0.4840 - val_loss: 1.1709

Epoch 8/20

8/8 ————— 3s 320ms/step - accuracy: 0.7087 -
loss: 0.7559 - val_accuracy: 0.6330 - val_loss: 0.9625

Epoch 9/20

8/8 ————— 3s 322ms/step - accuracy: 0.6693 -
loss: 0.8647 - val_accuracy: 0.7660 - val_loss: 0.7364

Epoch 10/20

8/8 ————— 3s 331ms/step - accuracy: 0.6811 -
loss: 0.7449 - val_accuracy: 0.6383 - val_loss: 0.9032

Epoch 11/20

8/8 ————— 3s 331ms/step - accuracy: 0.7283 -
loss: 0.6665 - val_accuracy: 0.9149 - val_loss: 0.5702

Epoch 12/20

8/8 ————— 3s 319ms/step - accuracy: 0.7323 -
loss: 0.6142 - val_accuracy: 0.9309 - val_loss: 0.4594

Epoch 13/20

8/8 ————— 3s 316ms/step - accuracy: 0.7795 -
loss: 0.5455 - val_accuracy: 0.9043 - val_loss: 0.4182

Epoch 14/20

8/8 ————— 3s 323ms/step - accuracy: 0.7441 -
loss: 0.6118 - val_accuracy: 0.9574 - val_loss: 0.2711

Epoch 15/20

8/8 ————— 3s 327ms/step - accuracy: 0.7717 -
loss: 0.5973 - val_accuracy: 0.9468 - val_loss: 0.2557

Epoch 16/20

8/8 ————— 3s 339ms/step - accuracy: 0.7795 -
loss: 0.5575 - val_accuracy: 0.9468 - val_loss: 0.2317

Epoch 17/20

8/8 ————— 3s 346ms/step - accuracy: 0.7598 -
loss: 0.5379 - val_accuracy: 0.9681 - val_loss: 0.1824

Epoch 18/20

8/8 ————— 3s 344ms/step - accuracy: 0.7795 -
loss: 0.5271 - val_accuracy: 0.9681 - val_loss: 0.1402

Epoch 19/20

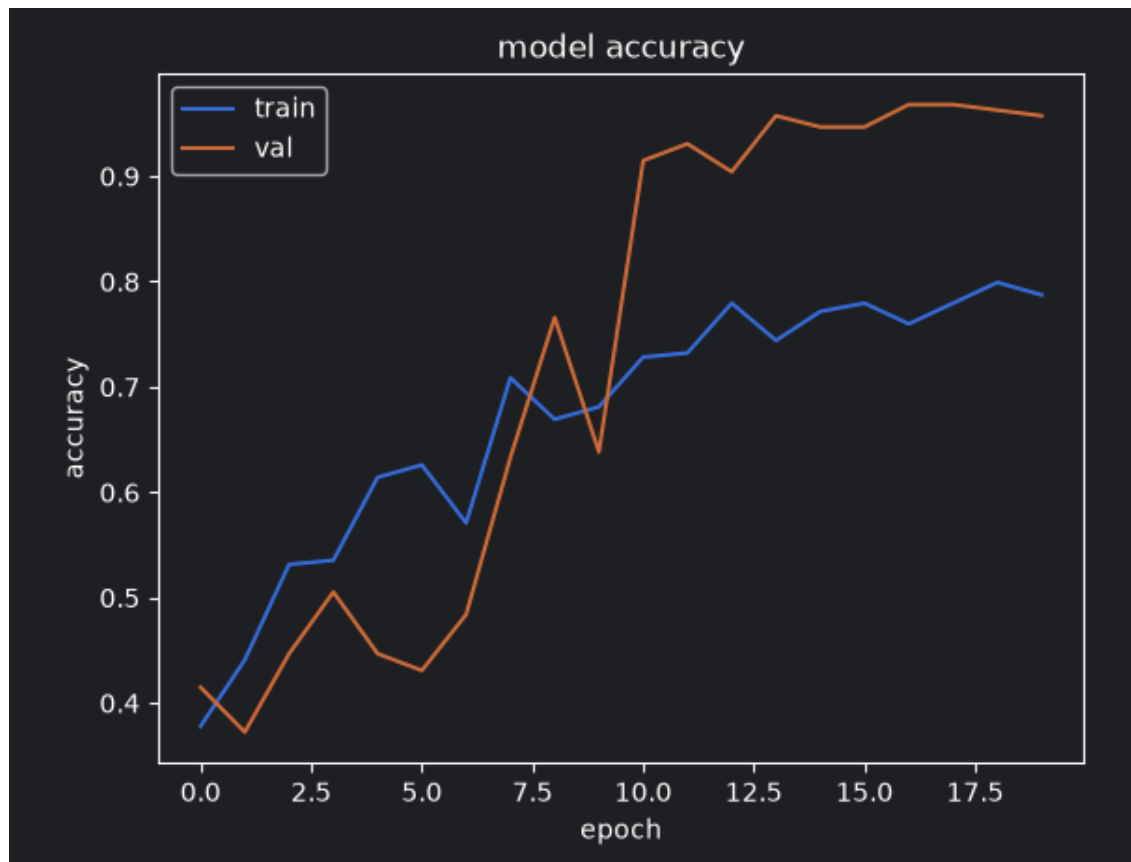
8/8 ————— 3s 353ms/step - accuracy: 0.7992 -
loss: 0.4928 - val_accuracy: 0.9628 - val_loss: 0.1388

Epoch 20/20

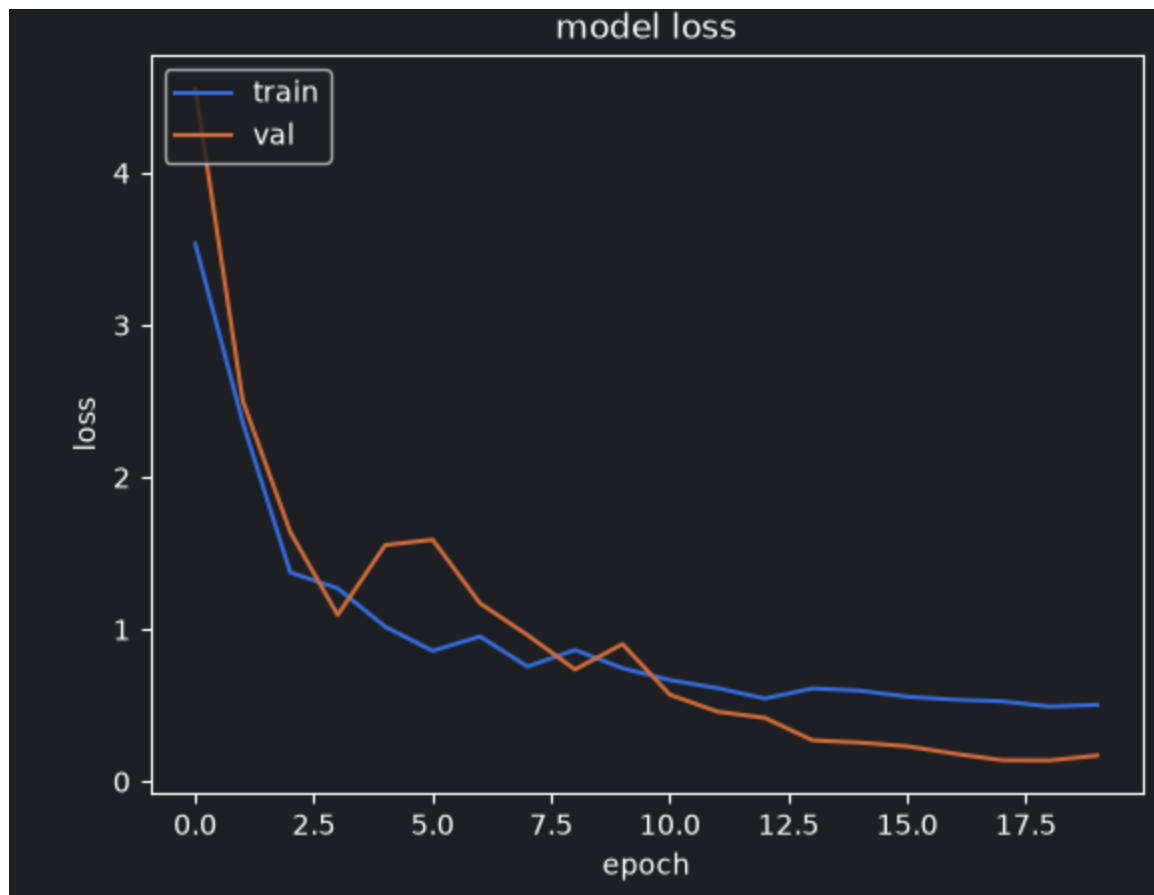
8/8 ————— 3s 328ms/step - accuracy: 0.7874 -
loss: 0.5042 - val_accuracy: 0.9574 - val_loss: 0.1706

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



```
plt.plot(history.history['loss'])
plt.plot(history.history['val loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



```
val_loss, val_accuracy = model.evaluate(val_ds)
print(f"Validation Accuracy: {val_accuracy:.4f}")
```

6/6 ————— 0s 55ms/step - accuracy:
0.9574 - loss: 0.1706

Validation Accuracy: 0.9574

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

# Get class names
class_names = val_ds.class_names

# Step 1: Collect true and predicted labels
y_true = []
y_pred = []
```

```

for images, labels in val_ds:
    preds = model.predict(images)
    preds = np.argmax(preds, axis=1) # get class with highest probability

    y_true.extend(labels.numpy())
    y_pred.extend(preds)

y_true = np.array(y_true)
y_pred = np.array(y_pred)

```

1/1 ————— 0s 190ms/step

1/1 ————— 0s 69ms/step

1/1 ————— 0s 71ms/step

1/1 ————— 0s 80ms/step

1/1 ————— 0s 77ms/step

1/1 ————— 0s 171ms/step

```
print(val_ds.class_names)
```

['apple', 'avocado', 'blueberry', 'capsicum']

```

print("Classification Report:")

print(classification_report(
    y_true,
    y_pred,
    labels=[0,1],
    target_names=class_names
))

```

Classification Report:

	precision	recall	f1-score	support
apple	0.60	0.86	0.71	7
avocado	0.33	0.20	0.25	5
micro avg	0.54	0.58	0.56	12
macro avg	0.47	0.53	0.48	12
weighted avg	0.49	0.58	0.52	12

```
# Step 2: Print classification report
print("Classification Report:")
print(classification_report(y_true, y_pred, target_names=class_names))

from sklearn.metrics import confusion_matrix
```

```
print(train_ds)
```

```
import numpy as np

print("Class Names:", train_ds.class_names)
print("Unique y_true:", np.unique(y_true))
print("Unique y_pred:", np.unique(y_pred))
print("y_true shape:", y_true.shape)
print("y_pred shape:", y_pred.shape)
```

Class Names: ['apple', 'avocado', 'blueberry', 'capsicum']

Unique y_true: [0 1 2 3]

Unique y_pred: [0 1 2 3]

y_true shape: (188,)

y_pred shape: (188,)

```
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Create empty lists
y_true = []
y_pred = []
```

```

# Get prediction from validation dataset
for images, labels in val_ds:

    predictions = model.predict(images)

    predicted_classes = np.argmax(predictions, axis=1)

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_classes)

# Convert to numpy array
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

print(cm)

# Classification Report
print(
    classification_report(
        y_true,
        y_pred,
        target_names=train_ds.class_names
    )
)

# Plot Confusion Matrix
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=train_ds.class_names,
    yticklabels=train_ds.class_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()

```

1/1 ————— 0s 93ms/step

1/1 ————— 0s 71ms/step

1/1 ————— 0s 73ms/step

1/1 ————— 0s 76ms/step

1/1 ————— 0s 80ms/step

1/1 ————— 0s 77ms/step

[[6 1 0 0]

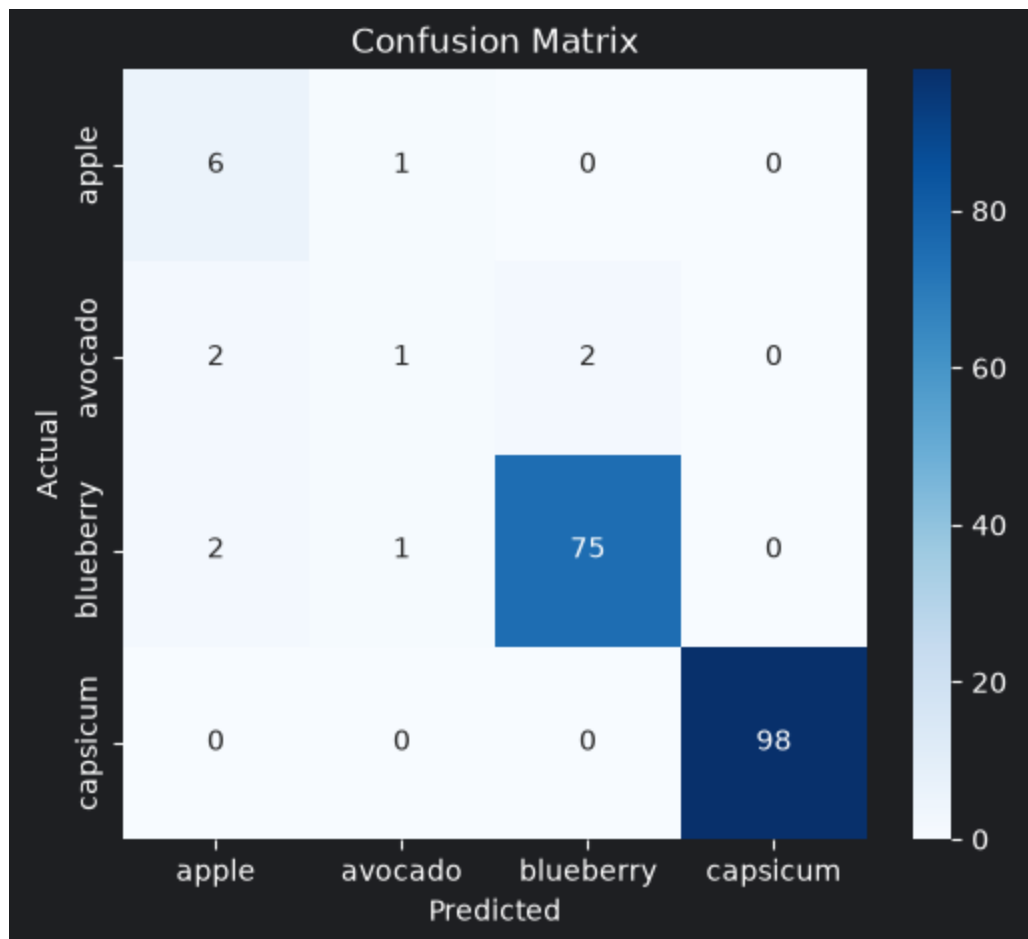
[2 1 2 0]

[2 1 75 0]

[0 0 0 98]]

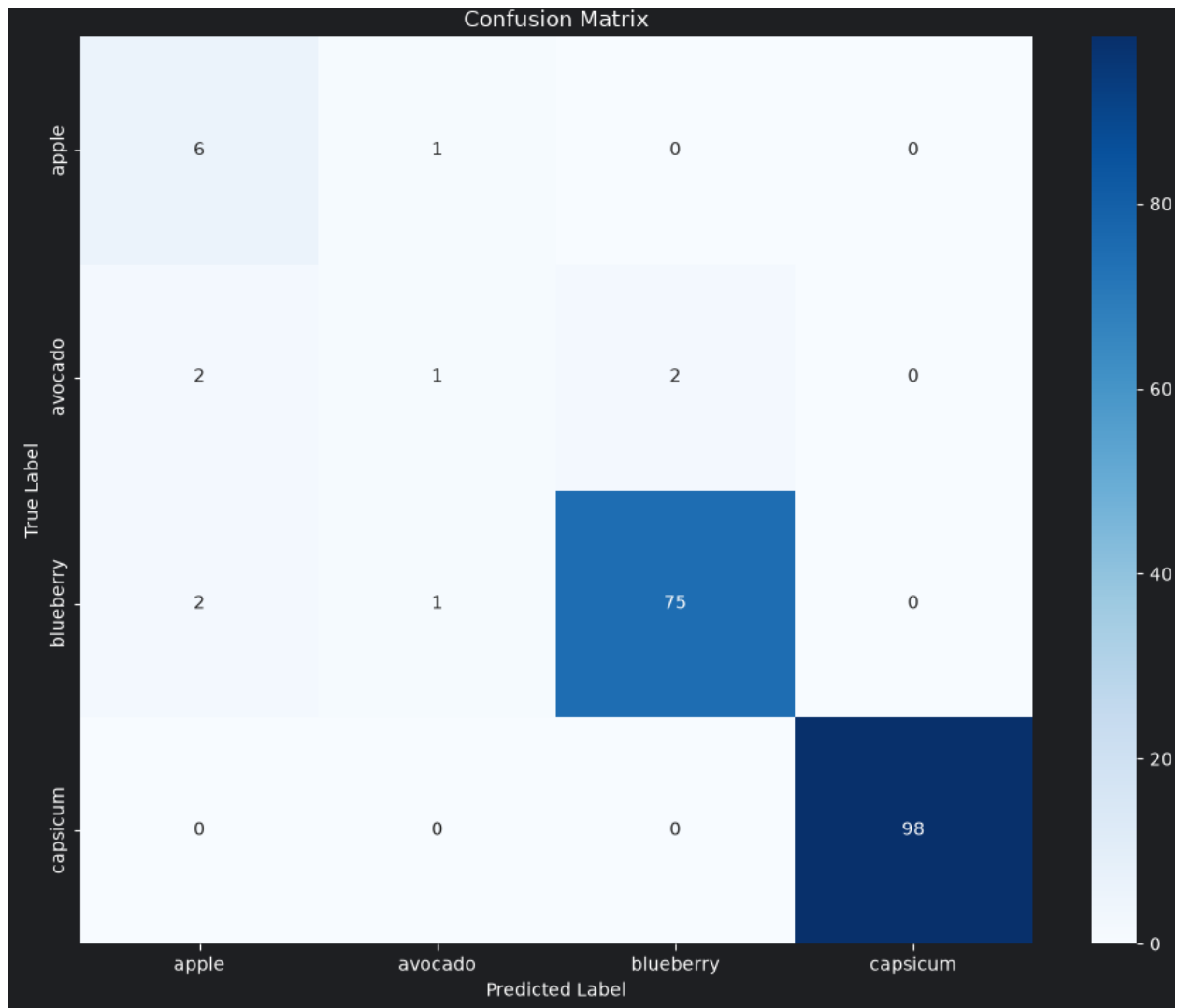
precision recall f1-score support

apple	0.60	0.86	0.71	7
avocado	0.33	0.20	0.25	5
blueberry	0.97	0.96	0.97	78
capsicum	1.00	1.00	1.00	98
accuracy		0.96		188
macro avg	0.73	0.75	0.73	188
weighted avg	0.96	0.96	0.96	188



```
cm = confusion_matrix(y_true, y_pred)

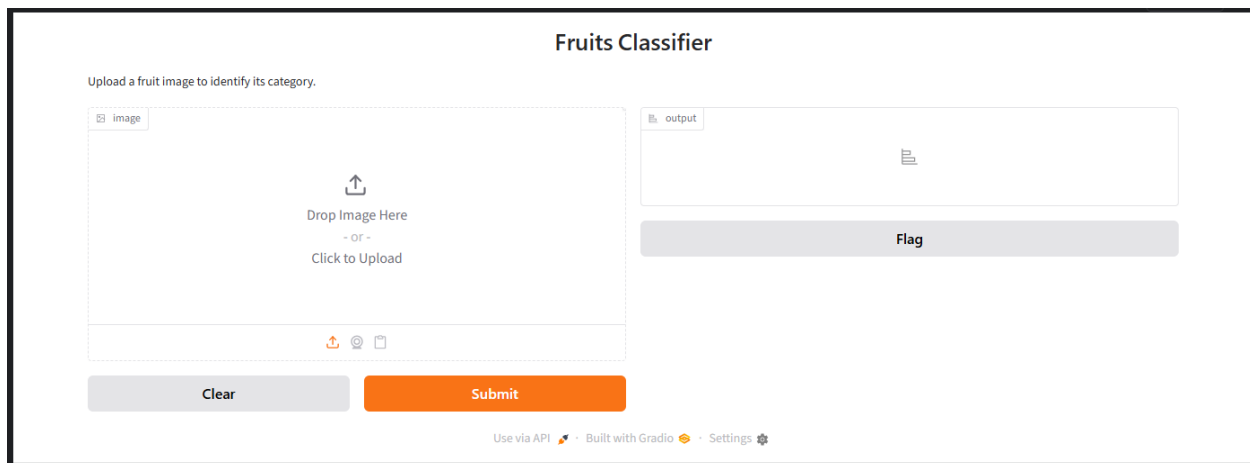
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



5. Graphical User Interface (GUI)

GUI Description

A Gradio-based web interface was developed to allow users to upload fruit images and receive classification results instantly.



GUI Features

- Upload fruit image
- Predict fruit category
- Display confidence scores
- Simple and user-friendly interface

GUI Implementation

```
import gradio as gr
import tensorflow as tf
import numpy as np

# Prediction function
def predict(image):
    # Resize and normalize image
    img = tf.image.resize(image, (128, 128)) # match model input size
    img = tf.expand_dims(img, axis=0) # add batch dimension

    # Predict
    pred = model.predict(img)
    class_idx = np.argmax(pred[0])
    class_name = class_names[class_idx]
    confidence = float(pred[0][class_idx])

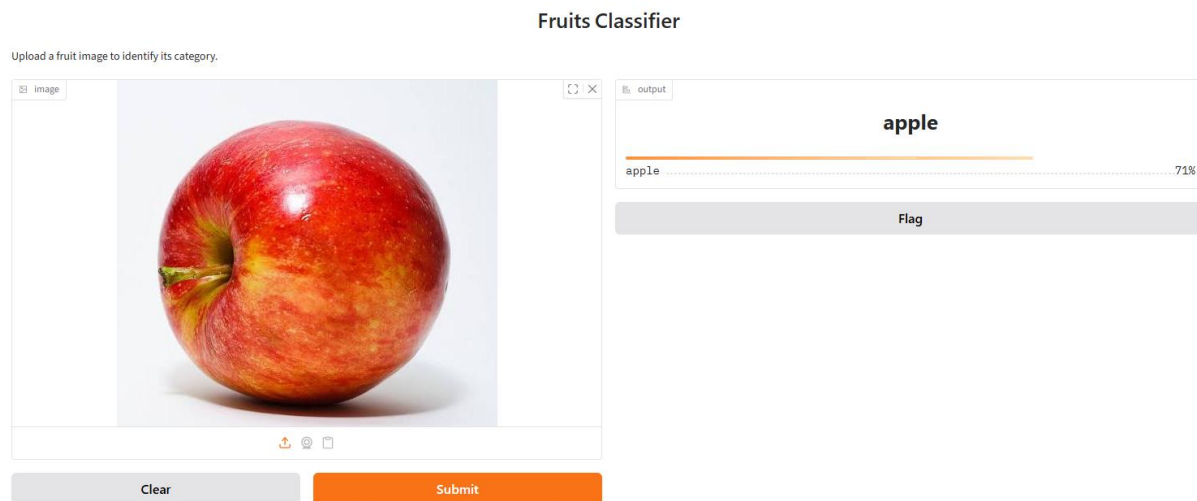
    return {class_name: confidence}

# Gradio interface
interface = gr.Interface(
    primary_hue="green",
    secondary_hue="emerald",
    fn=predict,
    inputs=gr.Image(type="numpy"),
    outputs=gr.Label(num_top_classes=3),
    title="Fruits Classifier",
```

```
        description="Upload a fruit image to identify its category."  
    )  
  
    # Launch the app  
    interface.launch()
```

Input Process

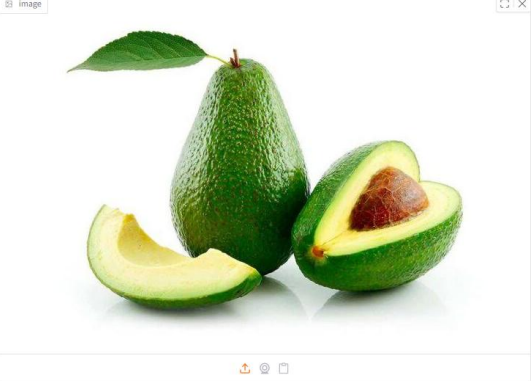
1. User uploads an image.
2. Image is preprocessed.
3. CNN model predicts the class.
4. Result is displayed.



Fruits Classifier

Upload a fruit image to identify its category.

image



Clear Submit

output

blueberry

blueberry

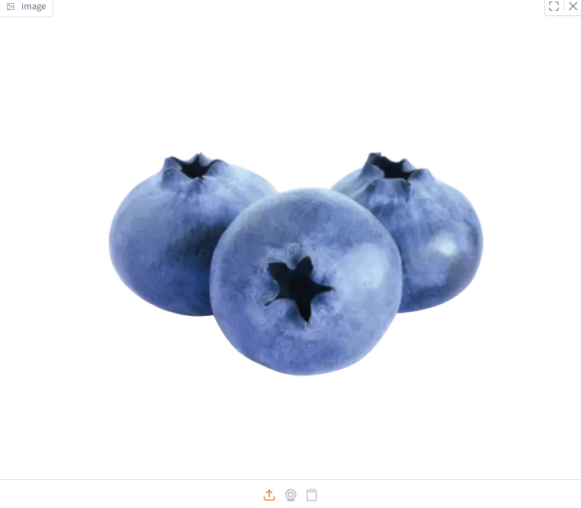
70%

Flag

Fruits Classifier

Upload a fruit image to identify its category.

image



Clear Submit

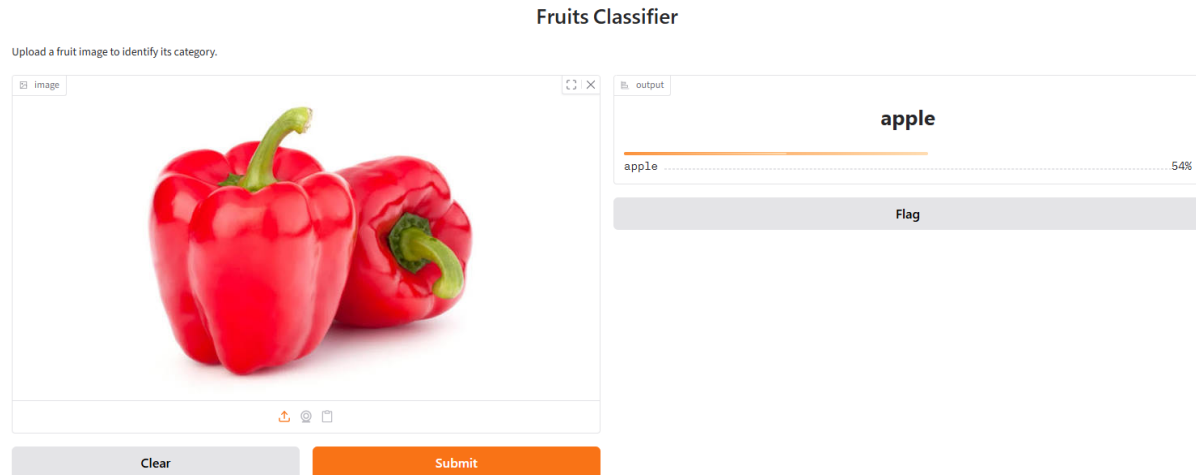
output

blueberry

blueberry

100%

Flag



Uploaded Image	Prediction
Apple Image	Apple (71%)
Avocado Image	Avocado (70%)
Blueberry Image	Blueberry (100%)
Capsicum	Apple (54%)

Conclusion

The proposed CNN-based fruit classification system successfully identifies Apple, Avocado, Blueberry, and Capsicum images with high accuracy. The developed GUI provides an easy-to-use platform for real-time fruit recognition, making the system suitable for agricultural and industrial applications.